



Computer Architecture

DEVELOPMENT REPORT

ARES - ARES Recursive Experimental System
Technical Implementation and Architecture Documentation

Authors:

Juan Ignacio Raggio

Enzo Canelo

Matias Sanchez

Second Semester 2025

18/06/2026

Table of Contents

1	Introduction	4
1.1	Project Overview	4
1.2	Scope of Implementation	4
1.3	Development Philosophy	4
2	Directory Structure Reorganization	5
2.1	Original x64BareBones Structure	5
2.2	AresOS Reorganized Structure	5
2.3	Benefits of Reorganization	6
2.3.1	Separation of Concerns	6
2.3.2	Improved Maintainability	6
2.3.3	Scalability	6
2.3.4	Build System Clarity	6
3	Interesting Implementation Techniques	7
3.1	Structs in Assembly	7
3.1.1	Register Snapshot Structure	7
3.1.2	Advantages of Assembly Structs	8
3.2	Unions for Shell Command Dispatch	9
3.2.1	Command Function Abstraction	9
3.2.2	Type-Safe Dispatch Mechanism	9
3.2.3	Benefits of Union-Based Dispatch	10
4	Userland-Kernel Communication	11
4.1	System Call Mechanism: SYSCALL vs INT 0x80h	11
4.1.1	Research and Comparison	11
4.1.2	Decision: SYSCALL for AresOS	12
4.2	SYSCALL Implementation Details	13
4.2.1	MSR Initialization	13
4.2.2	Userland System Call Stub	13
4.2.3	Kernel System Call Entry Point	15
4.2.4	SYSCALL Execution Flow	15
4.2.5	System Call Table	17
4.2.6	Challenges with Return Values and Context Switching	17
4.2.7	Example System Call Handler	18
4.3	Interrupt and IRQ Handling	20
4.3.1	Interrupt Descriptor Table (IDT)	20
4.3.2	IRQ Handler Structure	20
4.3.3	IRQ Dispatcher	20
4.4	Exception Handling	22
4.4.1	Exception Handler Macro	22
4.4.2	Exception Dispatcher and Register Snapshot	22
5	Video System and RGB Color Support	23
5.1	Evolution from VGA to RGB	23
5.1.1	RGB Color System	23
5.2	RGB Implementation Details	23
5.2.1	ncPrintCharRGB Function	23
5.2.2	System Call Integration	24
5.3	User-Facing Color Commands	25

5.3.1	textcolor Command	25
5.3.2	bgcolor Command	25
5.4	Benefits of RGB System	26
5.4.1	User Customization	26
5.4.2	Modern Graphics	26
5.4.3	Extensibility	26
5.4.4	Backward Compatibility	26
6	Audio System	27
6.1	PC Speaker Driver	27
6.1.1	Hardware Interface	27
6.1.2	System Call Interface	27
6.1.3	Applications	28
7	Performance Benchmarks	29
7.1	Benchmark Suite	29
7.1.1	FPS (Frames Per Second) Benchmark	29
7.1.2	Timer Benchmark	29
7.1.3	Syscall Read Benchmark	29
7.2	Benchmark Results	30
7.3	Macbook Pro M3 - QEMU Virtualization	30
7.4	Surface 4 - QEMU Virtualization	31
7.4.1	Cross-Platform Consistency	31
8	Code Quality and Extensibility	32
8.1	Emphasis on Code Quality	32
8.1.1	Clean Code Principles	32
8.1.2	Modular Architecture	32
8.1.3	Type Safety	32
8.2	Extensibility by Design	32
8.2.1	Adding New System Calls	32
8.2.2	Adding New Shell Commands	33
8.2.3	Adding New Exception Handlers	33
8.2.4	Adding New Drivers	34
8.3	Benefits of This Approach	34
8.4	Key Achievements	35
8.4.1	Architectural Decisions	35
8.4.2	Technical Implementation	35
8.4.3	Code Quality	35

1 Introduction

1.1 Project Overview

ARES (ARES Recursive Experimental System) is an educational operating system developed for the x86-64 architecture. This report documents the technical implementation details of the Kernel and Userland components that were developed on top of the x64BareBones baseline project.

The x64BareBones project provided the foundational bootloader and basic system initialization. Our team's primary contribution consisted of implementing a complete kernel with system call infrastructure, interrupt handling, exception management, and a fully-featured userland environment with an interactive shell and applications.

1.2 Scope of Implementation

Important: Our implementation focused exclusively on the Kernel and Userland components. The bootloader (Pure64) and basic hardware initialization were provided by the x64BareBones baseline. This allowed us to concentrate on operating system design principles, privilege level transitions, and user-kernel communication mechanisms.

The key areas of our development include:

- **Kernel Architecture:** Complete reorganization and implementation of kernel subsystems
- **System Call Interface:** Modern SYSCALL/SYSRET mechanism for x86-64
- **Interrupt and Exception Handling:** Comprehensive IDT setup and handler implementation
- **Userland Environment:** Shell, C library, and user applications
- **Driver Development:** Video, keyboard, timer, and RTC drivers

1.3 Development Philosophy

Throughout this project, we prioritized two fundamental principles:

Code Quality Over Product Features: We deliberately chose to write clean, well-structured, and maintainable code rather than implementing a large number of features. This decision makes the codebase more understandable, easier to debug, and significantly simpler to extend in the future.

Extensibility and Modularity: Every subsystem was designed with future expansion in mind. The modular architecture allows new system calls, drivers, and shell commands to be added with minimal changes to existing code. This design philosophy is evident in our use of function tables, clean abstractions, and separation of concerns.

2 Directory Structure Reorganization

2.1 Original x64BareBones Structure

The x64BareBones baseline provided a simple, flat directory structure suitable for a minimal kernel. This structure works well for small projects but becomes difficult to maintain as complexity grows. All kernel C files resided in the root directory, and the include directory had no organization by subsystem.

2.2 AresOS Reorganized Structure

We significantly reorganized the project to improve maintainability, scalability, and logical separation of concerns. This approach forced us to change almost all Makefiles provided and the build system, even though this brought us some problems, the maintainability and organization of the project improved significantly

```
AresOS/
├── Bootloader/           # Unchanged from baseline
├── Kernel/
│   ├── arch/            # Architecture-specific code
│   │   └── x86_64/
│   │       ├── interrupts/
│   │       │   ├── interrupts.asm
│   │       │   ├── syscalls.c
│   │       │   └── exceptions.c
│   │   ├── asm/         # Low-level assembly
│   │       ├── init_syscalls.asm
│   │       ├── syscalls.asm
│   │       └── syscalls_table.asm
│   ├── core/            # Core kernel functionality
│   ├── drivers/         # Hardware drivers
│   │   ├── keyboard/
│   │   ├── timer/
│   │   └── video/
│   ├── include/         # Headers organized by subsystem
│   │   ├── arch/
│   │   │   └── x86_64/
│   │   ├── core/
│   │   └── drivers/
│   ├── lib/             # Kernel utility libraries
│   └── build/           # Build artifacts (organized)
└── Userland/
    ├── UserCodeModule/
    │   ├── asm/         # User-level assembly (syscall stubs)
    │   ├── include/     # User headers
    │   ├── libc/        # C library implementation
    │   └── src/         # Application source code
    │       ├── shell.c
    │       ├── commands.c
    │       └── ...
```

2.3 Benefits of Reorganization

The reorganized structure provides several significant advantages:

2.3.1 Separation of Concerns

Each directory has a clear, well-defined purpose. Architecture-specific code is isolated in `arch/x86_64/`, making it straightforward to add support for other architectures in the future. Drivers are organized by device type, and core kernel functionality is separated from hardware-specific code.

2.3.2 Improved Maintainability

When searching for code related to a specific subsystem (e.g., interrupts, keyboard driver, system calls), it's easier to know exactly where to look. This organization makes the project significantly easier to understand.

2.3.3 Scalability

As new features are added, they have clear homes in the directory structure. Adding a new driver means creating a new subdirectory under `drivers/`. Adding a new architecture means creating a new subdirectory under `arch/`.

2.3.4 Build System Clarity

The organized structure allows for cleaner, more maintainable Makefiles. Dependencies are easier to track, and partial builds of specific subsystems become straightforward, so if an asm file is added to the project, it's trivial to understand which Makefile should be changing.

3 Interesting Implementation Techniques

3.1 Structs in Assembly

One of the notable techniques employed in AresOS is the use of structured data definitions directly in assembly language. This approach provides type safety and clarity when working with complex register snapshots and stack frames.

3.1.1 Register Snapshot Structure

In `AresOS/Kernel/arch/x86_64/interrupts/interrupts.asm`, we define a structured layout for CPU register snapshots:

```
struc regs
    _rip: resq 1
    _rsp: resq 1
    _rax: resq 1
    _rbx: resq 1
    _rcx: resq 1
    _rdx: resq 1
    _rbp: resq 1
    _rdi: resq 1
    _rsi: resq 1
    _r8:  resq 1
    _r9:  resq 1
    _r10: resq 1
    _r11: resq 1
    _r12: resq 1
    _r13: resq 1
    _r14: resq 1
    _r15: resq 1
    _cs:  resq 1
    _rflags: resq 1
    _ss:  resq 1
endstruc
```

Then to save registers to the buffer, we use an elegant way of moving qwords (64 bits) for each register

```
exceptionHandler 1
    cli
    pushState
    ; Copy registers from stack to regs_buffer safely
    mov rsi, rsp          ; Source = stack
    lea rdi, [regs_buffer] ; Destination = buffer
    mov rcx, 20           ; 20 registers (qwords)
    rep movsq             ; Copy efficiently

    ; Call exceptionDispatcher
    mov rdi, %1           ; Exception number
    lea rsi, [regs_buffer] ; Pointer to safe copy

    call exceptionDispatcher
```

3.1.2 Advantages of Assembly Structs

Using structs in assembly provides several benefits:

- **Type Safety:** Field offsets are calculated automatically by the assembler, eliminating manual offset calculations and reducing human errors and unreadable code
- **Maintainability:** Adding or reordering fields requires changes only to the struct definition, you don't need to initialize the structure in order
- **Interoperability:** The struct layout matches the corresponding C struct, ensuring seamless data exchange between assembly and C code
- **Readability:** The possibility of using named fields (`regs + _rax`) is much clearer than magic number offsets

3.2 Unions for Shell Command Dispatch

The shell implementation demonstrates sophisticated use of C union types to create a flexible, type-safe command dispatch system that can handle commands with varying numbers of parameters. This approach is inspired by lambda functions in Java

3.2.1 Command Function Abstraction

In `AresOS/Userland/UserCodeModule/include/commands.h`, we define a union that can hold function pointers with different signatures:

```
typedef enum {
    supplier_t = 0,      // No parameters
    function_t,          // One parameter
    bi_function_t,       // Two parameters
    max_parameters,
} function_type;

typedef union {
    int (*supplier)(void);           // Commands like 'help', 'clear'
    int (*function)(char *);        // Commands like 'man <cmd>'
    int (*bi_function)(char *, char *); // Commands like 'div <a> <b>'
} executable_t;

typedef struct {
    executable_t execute;
    function_type ftype;
} lambda_t;
```

Each command is then defined with its appropriate function type:

```
static const command_t div_command = {
    .name      = "div",
    .description = "Integer division of two numbers",
    .lambda = {
        .execute.bi_function = (void *)&div_cmd,
        .ftype               = bi_function_t,
    },
};

static const command_t help_command = {
    .name      = "help",
    .description = "List all available commands",
    .lambda = {
        .execute.supplier = (void *)&help,
        .ftype            = supplier_t,
    },
};
```

3.2.2 Type-Safe Dispatch Mechanism

This design enables type-safe runtime dispatch based on the number of parameters. The shell can validate parameter counts and call the appropriate function with compile-time type checking:

```
// Pseudo-code for command execution
switch (cmd->lambda.ftype) {
    case supplier_t:
        result = cmd->lambda.execute.supplier();
        break;
    case function_t:
        result = cmd->lambda.execute.function(arg1);
        break;
    case bi_function_t:
        result = cmd->lambda.execute.bi_function(arg1, arg2);
        break;
}
```

3.2.3 Benefits of Union-Based Dispatch

This approach provides several advantages over traditional function pointer tables:

- **Extensibility:** Adding commands with different parameter counts requires no changes to the dispatch logic
- **Type Safety:** The compiler ensures that each function pointer matches its declared signature
- **Self-Documenting:** The function type explicitly declares how many parameters a command expects
- **No Variable-parameter Functions:** Avoids the type unsafety and runtime overhead of functions with variable parameters

4 Userland-Kernel Communication

Important: The communication mechanism between Userland and Kernel is the key of our operating system architecture. We invested significant research and development effort into designing a robust, efficient, and modern system call interface. We found out the usage of the `syscall` instruction over the `int 0x80h` is most commonly used in 64-bits systems and the `int 0x80h` provided by the course is mostly used in 32-bits systems

4.1 System Call Mechanism: SYSCALL vs INT 0x80h

One of the most critical architectural decisions in AresOS was choosing between the traditional `INT 0x80h` software interrupt mechanism and the modern `SYSCALL` instruction for implementing system calls in 64-bit mode.

4.1.1 Research and Comparison

We conducted extensive research comparing both mechanisms across multiple dimensions:

4.1.1.1 INT 0x80h (Software Interrupt)

Advantages:

- Universal compatibility across x86 and x86-64
- Simpler conceptual model (just another interrupt)
- Familiar to developers from 32-bit systems
- Main approach used during the course

Disadvantages:

- Significant performance overhead
 - Full interrupt dispatch through IDT
 - Stack switching through TSS
 - Segment register checks and validation
- Requires IDT entry setup
- Not optimized for 64-bit mode
- Approximately 300-1000+ CPU cycles per invocation (source: Linux kernel documentation on syscall performance)

4.1.1.2 SYSCALL/SYSRET (Fast System Call)

Advantages:

- Designed specifically for x86-64 long mode
- Minimal overhead (approximately 50-150 cycles)
- Direct jump to kernel handler without IDT lookup
- Automatic privilege level switching
- Hardware-optimized stack and register management
- Uses Model-Specific Registers (MSRs) for configuration

Disadvantages:

- Requires explicit MSR initialization
- x86-64 specific (not portable to 32-bit)
- Slightly more complex initial setup

- This approach was not covered in the course, so unexpected issues may arise

4.1.2 Decision: SYSCALL for AresOS

After this analysis, we chose SYSCALL/SYSRET for the following reasons:

1. **Performance:** In a 64-bit operating system, system calls are frequent operations. The 2-3x performance improvement translates to better overall system responsiveness.
2. **Modern Design:** SYSCALL is the intended mechanism for system calls in 64-bit mode. Using it aligns with modern operating system design principles.
3. **Cleaner Semantics:** SYSCALL has well-defined register conventions specifically for system calls, unlike INT which is a general-purpose interrupt mechanism.
4. **Future-Proof:** Modern operating systems (Linux, BSD, Windows) all use SYSCALL on x86-64, making our implementation more aligned with industry standards.

Note: While SYSCALL is x86-64 specific, this is not a limitation for AresOS, which targets exclusively the x86-64 architecture. If portability to 32-bit systems becomes a requirement, a separate INT 0x80h implementation could be added alongside the SYSCALL interface but the outstanding project structure makes this changes simple.

4.2 SYSCALL Implementation Details

Our SYSCALL implementation consists of three main components: MSR initialization, userland stubs, and kernel handlers.

4.2.1 MSR Initialization

The SYSCALL mechanism requires configuring three Model-Specific Registers. This initialization happens during kernel boot in `AresOS/Kernel/asm/init_syscalls.asm`:

```
init_syscalls:
    ; 1. Enable SYSCALL in EFER (Extended Feature Enable Register)
    mov ecx, 0xC0000080      ; IA32_EFER
    rdmsr
    or eax, 1                ; Bit 0 = SCE (System Call Extensions)
    wrmsr

    ; 2. STAR: Configure segment selectors
    ; Bits 63:48 = User CS (0x1B for ring 3)
    ; Bits 47:32 = Kernel CS (0x08 for ring 0)
    mov ecx, 0xC0000081      ; IA32_STAR
    xor eax, eax              ; Bits 31:0 = 0 (unused)
    mov edx, 0x00180008      ; EDX = [User CS : Kernel CS]
    wrmsr

    ; 3. LSTAR: Set handler address (64-bit)
    mov ecx, 0xC0000082      ; IA32_LSTAR
    lea rax, [rel syscall_entry] ; Handler address
    mov rdx, rax
    shr rdx, 32               ; RDX = bits 63:32
    wrmsr

    ; 4. SFMASK: Set RFLAGS mask (optional)
    mov ecx, 0xC0000084
    xor eax, eax
    xor edx, edx
    wrmsr

    ret
```

This initialization:

- Enables SYSCALL support in the CPU
- Configures automatic segment selector switching (ring 3 to ring 0)
- Sets the kernel entry point for all system calls
- Optionally masks RFLAGS bits during transition

4.2.2 Userland System Call Stub

User applications invoke system calls through a simple assembly stub in `AresOS/Userland/UserCodeModule/asm/syscall.asm`:

```
global _syscall3

section .text
_syscall3:
    ; Arguments (System V AMD64 ABI):
    ; RDI = syscall number
    ; RSI = arg1
    ; RDX = arg2
    ; RCX = arg3

    mov rax, rdi    ; Move syscall number to RAX
    mov rdi, rsi    ; Shift arg1 to RDI
    mov rsi, rdx    ; Shift arg2 to RSI
    mov rdx, rcx    ; Shift arg3 to RDX
    syscall         ; Enter kernel
    ret             ; Return with result in RAX
```

This stub follows the System V AMD64 calling convention for user functions but adapts it to the SYSCALL register convention (syscall number in RAX, arguments in RDI, RSI, RDX).

4.2.3 Kernel System Call Entry Point

When a SYSCALL instruction executes, control transfers to the kernel entry point in `AresOS/Kernel/asm/syscalls.asm`:

```
syscall_entry:
    swapgs                ; Switch to kernel GS (get kernel data)
    mov r15, rsp          ; Save user stack pointer
    mov rsp, kernel_stack_top ; Switch to kernel stack

    ; Build stack frame
    push r15              ; Save user RSP
    push rcx              ; Save user RIP (return address)
    push r11              ; Save user RFLAGS

    ; Save callee-saved registers
    push rbx
    push rbp
    push r12
    push r13
    push r14

    ; Dispatch to appropriate syscall handler
    mov rbx, rax          ; RBX = syscall number
    lea rcx, [rel syscalls_table]
    shl rbx, 3            ; rbx * 8 (pointer size)
    add rcx, rbx
    mov rbx, [rcx]        ; Load function pointer
    test rbx, rbx
    jz .unknown_syscall

    ; Arguments already in RDI, RSI, RDX (correct positions)
    call rbx              ; Call syscall handler
    jmp .done

.unknown_syscall:
    mov rax, -1           ; Error code

.done:
    ; Restore registers in reverse order
    pop r14
    pop r13
    pop r12
    pop rbp
    pop rbx
    pop r11              ; Restore user RFLAGS
    pop rcx              ; Restore user RIP
    pop rsp              ; Restore user RSP

    swapgs                ; Switch back to user GS
    sysretq              ; Return to userland
```

4.2.4 SYSCALL Execution Flow

The complete system call flow follows this sequence:

1. **Userland Application** calls a wrapper function (e.g., `write()`)
2. **Wrapper Function** calls `_syscall3` stub with syscall number and arguments
3. **SYSCALL Instruction** triggers privilege level transition (ring 3 $\rightarrow$ ring 0)
4. **CPU Automatically:**
 - Saves user RIP to RCX
 - Saves user RFLAGS to R11
 - Loads kernel RIP from LSTAR MSR
 - Switches to kernel code segment (from STAR MSR)
5. **Kernel Entry Point** (`syscall_entry`):
 - Switches to kernel stack
 - Saves user context (RSP, RIP, RFLAGS)
 - Saves callee-saved registers
 - Dispatches to appropriate handler
6. **Syscall Handler** executes in kernel mode, returns result in RAX
7. **Kernel Entry Point** restores user context
8. **SYSRETQ Instruction** returns to userland (ring 0 $\rightarrow$ ring 3)
9. **CPU Automatically:**
 - Restores user RIP from RCX
 - Restores user RFLAGS from R11
 - Switches to user code segment
10. **Userland** continues with result in RAX

4.2.5 System Call Table

System calls are dispatched through a function pointer table defined in `AresOS/Kernel/asm/syscalls_table.asm`:

```
section .data
global syscalls_table

syscalls_table:
    dq sys_write          ; 0: SYS_WRITE
    dq sys_exit           ; 1: SYS_EXIT
    dq sys_read_wrapper   ; 2: SYS_READ
    dq sys_clear_wrapper  ; 3: SYS_CLEAR
    dq sys_get_ticks_wrapper ; 4: SYS_GET_TICKS
    dq sys_get_resolution_wrapper ; 5: SYS_GET_RESOLUTION
    dq sys_get_register_array_wrapper ; 6: SYS_GET_REGISTER_ARRAY
    dq sys_set_font_size_wrapper ; 7: SYS_SET_FONT_SIZE
    dq sys_get_memory_wrapper ; 8: SYS_GET_MEMORY
    dq sys_draw_rect_wrapper ; 9: SYS_DRAW_RECT
    dq sys_get_seconds_wrapper ; 10: SYS_GET_SECONDS
    dq sys_set_text_color_wrapper ; 11: SYS_SET_TEXT_COLOR
    dq sys_set_bg_color_wrapper ; 12: SYS_SET_BG_COLOR
    dq sys_get_cursor_pos_wrapper ; 13: SYS_GET_CURSOR_POS
    dq sys_redraw_screen_wrapper ; 14: SYS_REDRAW_SCREEN
    dq sys_get_time_wrapper ; 15: SYS_GET_TIME
```

This table-driven approach makes adding new system calls trivial: simply add a new entry to the table and define the corresponding handler function.

4.2.6 Challenges with Return Values and Context Switching

During development, we encountered a critical issue with syscall return values. Initially, our `syscall_entry` implementation saved ALL registers including RAX, following the same pattern used for IRQ handlers:

```
; Initial (incorrect) implementation
syscall_entry:
    push rax          ; Saved RAX - this was the problem
    push rbx
    push rbp
    ; ...
    call handler      ; Handler returns value in RAX
    ; ...
    pop rbp
    pop rbx
    pop rax           ; Overwrites the return value with syscall number
    sysretq
```

This caused syscalls to return nonsensical values to userland. The handler would correctly compute and return a value in RAX, but the `pop rax` instruction would immediately overwrite it with the original syscall number that was pushed at entry.

Symptoms:

- `read()` always returned the syscall number instead of bytes read
- Return values appeared random or incorrect
- Debugging showed handlers executed correctly, but userland received wrong results

Root cause: We were treating syscalls like interrupts, saving and restoring all registers. This breaks the calling convention where RAX must carry the return value.

Solution: Remove RAX from the saved registers in `syscall_entry`. RAX is intentionally left unsaved so the handler's return value propagates to userland:

```
; Corrected implementation
syscall_entry:
    ; RAX NOT saved - it will carry return value
    push rbx
    push rbp
    push r12-r14
    ; ...
    call handler    ; Handler returns in RAX
    ; ...
    pop r12-r14
    pop rbp
    pop rbx
    sysretq        ; RAX carries return value to userland
```

This demonstrates an important distinction: IRQ handlers must preserve all registers to be transparent, while syscalls must preserve RAX as a communication channel for return values.

4.2.7 Example System Call Handler

System call handlers are implemented in C. Here's an example from `AresOS/Kernel/arch/x86_64/interrupts/syscalls.c`:

```
uint64_t sys_read(uint64_t fd, char *buf, uint64_t count) {
    if (fd != 0 || count == 0 || buf == NULL) {
        return 0;
    }
    if (buffer_has_next()) {
        buf[0] = buffer_next();
        return 1;
    }
    return 0;
}

uint64_t sys_write(uint64_t fd, const char *buf, uint64_t len) {
    if (fd != STDOUT && fd != STDERR) {
        return 0;
    }

    uint8_t color = (fd == STDOUT) ?
        current_stdout_color : current_stderr_color;

    for (uint64_t i = 0; i < len; i++) {
        ncPrintChar(buf[i], color);
    }
}
```

```
    return len;  
}
```

These handlers have full access to kernel resources and drivers while maintaining clear interfaces for userland applications.

4.3 Interrupt and IRQ Handling

In addition to system calls, the kernel must handle hardware interrupts (IRQs) for devices like the keyboard and timer. These use a different mechanism from SYSCALL.

4.3.1 Interrupt Descriptor Table (IDT)

Hardware interrupts are handled through the traditional IDT mechanism. Each interrupt vector has a handler registered in the IDT during kernel initialization.

4.3.2 IRQ Handler Structure

IRQ handlers follow a standardized pattern defined in AresOS/Kernel/arch/x86_64/interrupts/interrupts.asm:

```
%macro irqHandlerMaster 1
    pushState                ; Save all registers

    mov rdi, %1              ; IRQ number as first parameter
    mov rsi, rsp              ; Stack pointer (for register access)
    call irqDispatcher        ; Call C dispatcher

    ; Send EOI (End of Interrupt) to PIC
    mov al, 20h
    out 20h, al

    popState                 ; Restore all registers
    iretq                    ; Return from interrupt
%endmacro

; Timer interrupt (IRQ 0)
_irq00Handler:
    irqHandlerMaster 0

; Keyboard interrupt (IRQ 1)
_irq01Handler:
    irqHandlerMaster 1
```

The `pushState` and `popState` macros save and restore all general-purpose registers, ensuring that interrupted code can resume exactly where it left off.

4.3.3 IRQ Dispatcher

The assembly handlers call into a C dispatcher that routes interrupts to appropriate device drivers:

```
void irqDispatcher(uint64_t irq, uint64_t *stack_frame) {
    switch (irq) {
        case 0: // Timer
            timer_handler();
            break;
        case 1: // Keyboard
            keyboard_handler();
            break;
    }
}
```

```
        // Additional IRQs...  
    }  
}
```

This design separates low-level register management (assembly) from high-level device handling logic (C).

4.4 Exception Handling

Exceptions represent error conditions or special events during program execution (e.g., division by zero, invalid opcode)

4.4.1 Exception Handler Macro

Exception handlers are defined using a macro that captures complete register state as mentioned before:

```
%macro exceptionHandler 1
    cli
    pushState

    ; Copy registers from stack to regs_buffer safely
    mov rsi, rsp          ; Source = stack
    lea rdi, [regs_buffer] ; Destination = buffer
    mov rcx, 20           ; 20 registers (qwords)
    rep movsq             ; Copy efficiently

    ; Call exceptionDispatcher
    mov rdi, %1           ; Exception number
    lea rsi, [regs_buffer] ; Pointer to safe copy

    call exceptionDispatcher

    popState
    call getStackBase
    mov [rsp+24], rax      ; The StackBase
    mov rax, userland
    mov [rsp], rax        ; Overwrite return address

    sti
    iretq
%endmacro
```

4.4.2 Exception Dispatcher and Register Snapshot

The exception dispatcher in `AresOS/Kernel/arch/x86_64/interrupts/exceptions.c` provides detailed information about the exception:

```
void exceptionDispatcher(int exception, regs_snapshot_t *regs) {
    saved_regs = *regs;

    if (exception == ZERO_EXCEPTION_ID)
        zero_division(regs);
    else if (exception == INVALID_OPCODE_ID)
        invalid_opcode(regs);

    print_registers(regs);
}
```

5 Video System and RGB Color Support

5.1 Evolution from VGA to RGB

One of the significant enhancements made to AresOS was the transition from the original 8-bit VGA color palette system to a full 32-bit RGB color system.

5.1.1 RGB Color System

We implemented a comprehensive RGB color system throughout the entire video subsystem:

```
// RGB Colors (32-bit format: 0xRRGGBB)
#define BLACK      0x000000
#define WHITE      0xFFFFFFFF
#define RED        0xFF0000
#define GREEN      0x00FF00
#define BLUE       0x0000FF
#define YELLOW     0xFFFF00
#define CYAN       0x00FFFF
#define MAGENTA    0xFF00FF
// ...
```

5.2 RGB Implementation Details

5.2.1 ncPrintCharRGB Function

The core of the RGB system is the `ncPrintCharRGB` function in `naiveConsole.c`:

```
extern uint32_t current_bg_color;

void ncPrintCharRGB(char c, uint32_t rgb) {
    if (videoMode == 0) {
        /* In text mode, convert RGB to VGA approximation */
        uint8_t vga_color = VGA_WHITE;
        if (rgb == 0x000000)
            vga_color = VGA_BLACK;
        else if ((rgb & 0xFF0000) > 0x800000)
            vga_color = VGA_WHITE;
        ncPrintCharText(c, vga_color);
    } else {
        bmp_font_t *font = getFont();

        if (c == '\n') {
            // Handle newline with background color
            gfxCursorY += font->height * fontScale;
            if (gfxCursorY + font->height * fontScale >= SCREEN_HEIGHT) {
                clearScreen(current_bg_color);
                gfxCursorX = 0;
                gfxCursorY = 0;
            }
        }
        return;
    }
}
```

```

    }

    if (c == '\b') {
        // Handle backspace with background color
        if (gfxCursorX >= font->width * fontScale) {
            gfxCursorX -= font->width * fontScale;
            drawRect(gfxCursorX, gfxCursorY,
                    font->width * fontScale,
                    font->height * fontScale,
                    current_bg_color);
        }
        return;
    }

    // Draw character with RGB color
    drawChar(c, gfxCursorX, gfxCursorY, rgb, font);
    gfxCursorX += font->width * fontScale;
}
}

```

This function:

- Accepts full 32-bit RGB colors
- Handles text mode with VGA color approximation
- Uses RGB colors directly in graphics mode
- Respects the current background color for special characters

5.2.2 System Call Integration

The `sys_write` syscall was reimplemented in C to support RGB colors:

```

uint32_t current_bg_color = 0x000000;
static uint32_t current_stdout_color_rgb = 0xFFFFFF;
static uint32_t current_stderr_color_rgb = 0xFF0000;

uint64_t sys_write(uint64_t fd, const char *buf, uint64_t len) {
    if (fd != 1 && fd != 2) {
        return 0;
    }
    if (buf == NULL || len == 0) {
        return 0;
    }

    uint32_t color = (fd == 1) ?
        current_stdout_color_rgb : current_stderr_color_rgb;

    for (uint64_t i = 0; i < len; i++) {
        ncPrintCharRGB(buf[i], color);
    }

    return len;
}

```


Previously, `sys_write` was implemented in assembly and used VGA colors. The C implementation allows for:

- Direct RGB color support
- Cleaner code that's easier to maintain
- Integration with the global color state

5.3 User-Facing Color Commands

To expose RGB capabilities to users, we implemented two shell commands:

5.3.1 textcolor Command

Changes the foreground text color:

```
uint8_t textcolor_cmd(char *color) {
    uint32_t color_value = parse_color(color);

    if (color_value == 0xFFFFFFFF) {
        printf("Invalid color: %s\n", color);
        printf("Valid colors: black, white, red, green, blue, "
               "yellow, cyan, magenta, gray, lightgray, darkgray\n");
        printf("Or use hex format: 0xRRGGBB\n");
        return INVALID_INPUT;
    }

    shell_status.font_color = color_value;
    syscall_set_text_color(color_value, STDOUT);
    printf("Text color changed\n");
    return OK;
}
```

5.3.2 bgcolor Command

Changes the background color:

```
uint8_t bgcolor_cmd(char *color) {
    uint32_t color_value = parse_color(color);

    if (color_value == 0xFFFFFFFF) {
        printf("Invalid color: %s\n", color);
        printf("Valid colors: black, white, red, green, blue, "
               "yellow, cyan, magenta, gray, lightgray, darkgray\n");
        printf("Or use hex format: 0xRRGGBB\n");
        return INVALID_INPUT;
    }

    shell_status.background_color = color_value;
    syscall_set_bg_color(color_value);
    printf("Background color changed\n");
    return OK;
}
```

5.4 Benefits of RGB System

The RGB color system provides several advantages:

5.4.1 User Customization

Users can personalize their terminal experience with any color combination they prefer, not just the 16 VGA colors.

5.4.2 Modern Graphics

The system takes full advantage of modern graphics hardware capabilities

5.4.3 Extensibility

Future applications can use the full RGB color space for sophisticated graphics, UI elements, and visual feedback.

5.4.4 Backward Compatibility

The system still supports text mode through VGA color approximation, ensuring compatibility with older hardware or emulators.

6 Audio System

6.1 PC Speaker Driver

AresOS includes a PC speaker driver that provides basic audio output capabilities through the legacy PC speaker hardware.

6.1.1 Hardware Interface

The PC speaker is controlled through the Programmable Interval Timer (PIT) and port 0x61:

```
void playSound(uint64_t frequency, uint64_t duration_ms) {
    if (frequency == 0 || duration_ms == 0) {
        return;
    }

    set_speaker_frequency(frequency);
    enable_speaker();

    /* Wait for the specified duration */
    uint64_t start_ms = get_time_ms();
    uint64_t end_ms   = start_ms + duration_ms;
    while (get_time_ms() < end_ms)
        ;

    disable_speaker();
}
```

6.1.2 System Call Interface

Two syscalls expose audio functionality to userland:

6.1.2.1 SYS_PLAY_SOUND

Plays a tone at a specific frequency for a given duration:

```
uint64_t sys_play_sound(uint64_t frequency, uint64_t duration_ms) {
    playSound(frequency, duration_ms);
    return 0;
}
```

Usage from userland:

```
syscall_play_sound(440, 500); // Play 440 Hz (A4) for 500ms
```

6.1.2.2 SYS_BEEP

Plays a short beep at a specified frequency:

```
uint64_t sys_beep(uint64_t frequency) {  
    beep(frequency);  
    return 0;  
}
```

Usage from userland:

```
syscall_beep(880); // Play 880 Hz (A5) beep
```

6.1.3 Applications

The audio system can be used for:

- User feedback (beeps for errors or confirmations)
- Simple melodies or alert sounds
- Debugging (audio signals during boot or operation)
- Sound effects in Tron or other applications

Note: While the PC speaker is limited, it provides a simple, universally available audio interface that works on all x86 hardware.

7 Performance Benchmarks

Important: To evaluate AresOS performance across different platforms and virtualization technologies. These benchmarks test CPU performance, timer accuracy, syscall read responsiveness, and graphics rendering speed.

7.1 Benchmark Suite

AresOS includes an integrated benchmark suite accessible through the `benchmark` shell command. The suite tests three key subsystems:

7.1.1 FPS (Frames Per Second) Benchmark

Tests graphics rendering performance by:

- Drawing rectangles and characters
- Measuring frame rendering time
- Calculating average FPS over multiple iterations

7.1.2 Timer Benchmark

Tests timer accuracy and resolution by:

- Reading the RTC (Real-Time Clock)
- Measuring PIT (Programmable Interval Timer) ticks
- Calculating time differences over multiple samples
- Verifying timer interrupt frequency

This benchmark ensures that timing-critical operations (delays, timeouts, scheduling) work correctly.

7.1.3 Syscall Read Benchmark

Tests Sys Read performance by:

- Measuring detection latency
- Testing buffer handling
- Verifying scan code translation
- Measuring character processing time

7.2 Benchmark Results

7.3 Macbook Pro M3 - QEMU Virtualization

```
Resume:
=====
===== FPS Benchmark Result =====
Total test duration: 20862 ms
Total tests produced: 20
Total frames measured: 3294
Minimum FPS observed: 145
Maximum FPS observed: 178
Average FPS observed: 157

===== Timer Hardware Benchmark =====
Test duration: 1043 ms
Expected ticks: 18
Actual ticks: 19
Missed ticks: 0
Max jitter: 55000 us

===== Syscall Read Benchmark =====
Total characters read: 100
Minimum latency: 0 us
Maximum latency: 0 us
Average latency: 0 us

===== Benchmark Completed Successfully =====
```

Figure 1: Benchmark results on Macbook Pro M3 running QEMU

System Configuration:

- Host: Macbook Pro M3
- Virtualization: QEMU
- CPU Emulation: x86-64
- Graphics: QEMU VGA emulation

Note: QEMU on Apple Silicon requires CPU emulation (x86-64 on ARM64), which adds overhead compared to native execution. Despite this, performance remains acceptable

7.4 Surface 4 - QEMU Virtualization

```
Resume:
=====

===== FPS Benchmark Result =====
Total test duration: 20862 ms
Total tests produced: 20
Total frames measured: 2448
Minimum FPS observed: 4
Maximum FPS observed: 250
Average FPS observed: 117

===== Timer Hardware Benchmark =====
Test duration: 1043 ms
Expected ticks: 18
Actual ticks: 19
Missed ticks: 0
Max jitter: 55000 us

===== Syscall Read Benchmark =====
Total characters read: 100
Minimum latency: 0 us
Maximum latency: 0 us
Average latency: 0 us

===== Benchmark Completed Successfully =====
```

Figure 2: Benchmark results on Laptop Surface 4 running QEMU

System Configuration:

- Host: Laptop Surface 4
- Virtualization: QEMU
- CPU Emulation: x86_64
- Graphics: QEMU VGA emulation

7.4.1 Cross-Platform Consistency

The benchmarks verify that AresOS:

- Runs consistently across different platforms
- Maintains functional correctness regardless of host
- Scales appropriately with available resources

Note: All benchmark results are reproducible by running the `benchmark` command in the AresOS shell. The benchmark suite ensures consistent testing methodology across all platforms.

8 Code Quality and Extensibility

Important: Throughout the development of AresOS, we prioritized code quality and extensibility over feature completeness. This philosophy reflects our belief that a well-architected, maintainable codebase is more valuable than a feature-rich but poorly structured one.

8.1 Emphasis on Code Quality

Our commitment to code quality manifests in several ways throughout the project:

8.1.1 Clean Code Principles

- **Meaningful Names:** Functions, variables, and structures use descriptive names that clearly convey their purpose (e.g., `exceptionDispatcher`, `syscall_entry`, `irqHandlerMaster`)
- **Consistent Style:** We use automated formatting (clang-format) enforced through Git hooks
- **Comments and Documentation:** Complex algorithms and hardware interactions are thoroughly documented

8.1.2 Modular Architecture

The codebase is organized into clear subsystems with well-defined interfaces:

- **Kernel Subsystems:** Interrupts, syscalls, drivers, core functionality
- **Driver Abstraction:** Each driver has a consistent interface
- **Library Functions:** Reusable utilities separated from specific implementations

8.1.3 Type Safety

We leverage C's type system to catch errors at compile time:

- Strong typing for function pointers in command dispatch
- Structured definitions for register snapshots
- Enumerated types for syscall numbers and command indices

8.2 Extensibility by Design

Every major component of AresOS was designed to be extended with minimal code changes:

8.2.1 Adding New System Calls

Adding a new system call requires only three steps:

1. Define the syscall number in `syscalls_numbers.h`:

```
typedef enum {  
    SYS_WRITE = 0,  
    SYS_EXIT,  
    // ... existing syscalls ...  
    SYS_NEW_FEATURE, // Add here  
    SYS_MAX,  
} syscall_codes;
```


2. Implement the handler in `syscalls.c`:

```
uint64_t sys_new_feature(uint64_t arg1, uint64_t arg2) {
    // Implementation
    return result;
}
```

3. Add to syscall table in `syscalls_table.asm`:

```
syscalls_table:
    dq sys_write
    dq sys_exit
    ; ...
    dq sys_new_feature_wrapper
```

No changes to the syscall entry logic, dispatcher, or userland stub are required.

8.2.2 Adding New Shell Commands

The union-based command system makes adding commands equally straightforward:

1. Implement the command function:

```
int my_new_cmd(char *arg) {
    // Implementation
    return 0;
}
```

2. Define the command structure:

```
static const command_t mynew_command = {
    .name      = "mynew",
    .description = "Description of my new command",
    .lambda = {
        .execute.function = (void *)&my_new_cmd,
        .ftype            = function_t,
    },
};
```

3. Add to command table:

```
static const command_t *const commands[QTY_COMMANDS] = {
    &help_command,
    // ... existing commands ...
    &mynew_command,
};
```

The shell automatically handles dispatch, parameter validation, and help text generation.

8.2.3 Adding New Exception Handlers

Exception handling is similarly extensible:

1. Define exception handler in assembly:

```
_exception13Handler:
    exceptionHandler 13 ; General Protection Fault
```

2. Implement handler in C:

```
static void general_protection_fault(regs_snapshot_t *regs) {
    ncPrint("EXCEPTION 13: GENERAL PROTECTION FAULT\n", VGA_RED);
    // Additional handling
}
```

3. Add to dispatcher:

```
void exceptionDispatcher(int exception, uint64_t *stack_frame) {
    // ...
    else if (exception == 13)
        general_protection_fault(&regs);
}
```

4. Register in IDT during initialization.

8.2.4 Adding New Drivers

The driver architecture allows new hardware support to be added as self-contained modules:

1. Create driver directory: `drivers/new_device/`
2. Implement standard driver interface:
 - Initialization function
 - IRQ handler (if applicable)
 - Public API functions
3. Register driver during kernel initialization
4. Add to build system

The driver can then be used by both kernel and userland code through system calls.

8.3 Benefits of This Approach

- Easier Debugging
- Lower Maintenance Cost
- Educational Value

Note: AresOS serves as a learning resource. Clean, well-documented code is more effective for learning operating system concepts.

8.4 Key Achievements

8.4.1 Architectural Decisions

We made informed technical decisions based on research and analysis:

- Choosing SYSCALL over INT 0x80h for optimal performance in 64-bit mode
- Reorganizing the directory structure for improved maintainability
- Implementing a modular, extensible architecture

8.4.2 Technical Implementation

We successfully implemented core operating system components:

- MSR-based SYSCALL/SYSRET mechanism with proper privilege transitions
- Comprehensive interrupt and exception handling
- Device drivers for keyboard, video, timer, RTC, and PC speaker
- User-space environment with C library and interactive shell
- PC speaker audio system for sound feedback
- Integrated benchmark for performance evaluation

8.4.3 Code Quality

We prioritized maintainability and extensibility:

- Clean, well-documented code
- Modular architecture with clear separation of concerns
- Type-safe abstractions using structs and unions
- Extensible design patterns throughout
- Comprehensive error handling and debugging information